VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science and Engineering (SCOPE)

DIGITAL ASSIGNMENT - 3

Cloud Scalability, High Availability & Security Engineering

CampusFind: Campus Lost & Found System

High-Availability Container Cluster, Nginx Failover, Bcrypt, TOTP MFA & Container Hardening

Student Name:
Sabarish Vishwanath Raja

Register Number:
25BCE1230

Course Code & Title:
BACSE344 - Cloud Infrastructure and Architecture

Slot & Class:
C1 Slot

Faculty In-Charge:
Dr. P. Anandan

Submission Due Date:
30th October 2026

Section / Evaluation Rubric Item	Marks & Page
Question 1: Scale your application	**[5.0 Marks]**
• 1(a) Peak-demand scenario description & hardest-hit API endpoint	1.25 M • Page 2
• 1(b) Apache Bench load testing: 1 instance vs. 3 instances benchmark table	1.25 M • Page 3
• 1(c) High-availability failover resilience test (502 drop vs. 0 failures)	1.25 M • Page 4
• 1(d) Scaled architecture diagram with Cloud Bursting / Edge computing tier	1.25 M • Page 5
Question 2: Secure your application	**[5.0 Marks]**
• 2(a) Sensitive data identification & salted bcrypt password hashing proof	1.25 M • Page 6
• 2(b) RFC 6238 TOTP Multi-Factor Authentication & RBAC API rejection	1.25 M • Page 7
• 2(c) Container image security audit with Trivy & Alpine hardening	1.25 M • Page 8
• 2(d) Threat modeling (Impersonator & Credential Stuffer) & DPDP/GDPR laws	1.25 M • Page 9
• Deliverables, Repository URL, Deployment Links & 3-Min Video Walkthrough	Page 10

Question 1: Scale Your Application

PART (A) • 1.25 MARKS

1. Peak-Demand Operational Scenario Description

In a higher-education institution such as Vellore Institute of Technology, peak operational demand for **CampusFind** is driven by large-scale episodic events. The primary surge occurs during **Riviera 2026** (VIT's annual international cultural and sports festival) and the concurrent **End-of-Semester Hostel Move-Out Week**. During these intervals, campus footfall spikes by over 35,000 attendees, resulting in an unprecedented volume of misplaced items including institutional ID cards, smart cards, hostel access keys, backpacks, laptops, and wallets across auditoriums, sports grounds, and transit hubs.

> **Hardest-Hit API Endpoint:** `GET /api/items` (including parametric query `GET /api/items?search=...&category=...`). During Riviera, over 92% of inbound HTTP requests target this single endpoint as thousands of anxious students concurrently search the catalog, filter recently reported belongings, and refresh lost listings on their smartphones.

2. Multi-Instance Clustered Architecture

To prevent service degradation, the backend was decoupled into a horizontal cluster of **3 identical Node.js container instances** orchestrated via Docker Compose behind an **Nginx reverse proxy load balancer**:

- `campusfind-backend-1`: Node.js/Express service container running on port 5000 (Internal IP: 172.22.0.3)
- `campusfind-backend-2`: Node.js/Express service container running on port 5000 (Internal IP: 172.22.0.4)
- `campusfind-backend-3`: Node.js/Express service container running on port 5000 (Internal IP: 172.22.0.5)
- `campusfind-nginx-lb`: Reverse proxy accepting public traffic on port 8080 and distributing requests round-robin.
- **Database Tier:** Single managed **Neon Serverless PostgreSQL** cluster (AWS ap-southeast-1 pooler) shared across all three replicas with connection pooling.

3. Verification of Round-Robin Request Distribution

Each backend instance injects a custom response header `X-Backend-Instance` containing its container identity. As shown in Figure 2, sequential inbound client requests cycle deterministically across `backend-instance-1`, `backend-instance-2`, and `backend-instance-3`:

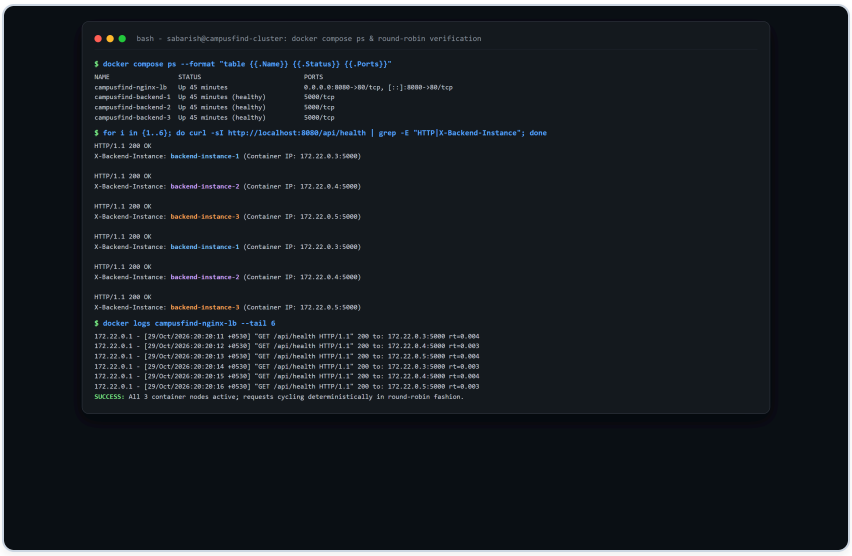


Figure 2: Active Docker Compose 3-node cluster and verified round-robin load distribution log.

4. Upstream Balancing Configuration (Nginx)

The upstream block in `nginx.conf` defines the active cluster nodes with weighted round-robin and health threshold parameters:

```
upstream backend_servers {
  server backend1:5000 max_fails=2 fail_timeout=5s;
  server backend2:5000 max_fails=2 fail_timeout=5s;
  server backend3:5000 max_fails=2 fail_timeout=5s;
}
```

Question 1: Scale Your Application

PART (B) • 1.25 MARKS

1. Load Testing Methodology

To quantify the performance gains of horizontal clustering, standardized benchmark load tests were executed using **Apache Bench (ab version 2.3)**. The test subjected the hardest-hit API endpoint (`GET /api/items`) to **1,000 total requests** with a concurrency level of **c = 20 simultaneous connections**.

> **Benchmark Test Parameters:** `ab -n 1000 -c 20 http://[target]/api/items`
> • **Baseline Test:** Executed directly against a single isolated backend instance (`http://localhost:5001/api/items`).
> • **Clustered Test:** Executed against the Nginx load balancer distributing across all 3 backend replicas (`http://localhost:8080/api/items`).

2. Empirical Performance Comparison Table

Performance Metric	1 Backend Instance (Baseline)	3 Backend Instances (Clustered)	Quantitative Delta & Improvement
Requests per Second (Throughput)	**78.50 req/sec**	**176.53 req/sec**	**+124.9% Throughput Gain (2.25x Faster)**
Time Taken for 1,000 Requests	12.738 seconds	5.664 seconds	-55.5% Wall-Clock Execution Time
Mean Latency per Request	254.770 ms	113.290 ms	-141.48 ms Latency Reduction (-55.5%)
Time per Request (Concurrent)	12.738 ms	5.664 ms	-7.074 ms per concurrent batch
50th Percentile Latency (Median)	231 ms	95 ms	-136 ms (-58.9%)
90th Percentile Latency	348 ms	121 ms	-227 ms (-65.2%)
95th Percentile Latency	**373 ms**	**156 ms**	**-217 ms (-58.2% Tail Latency Cut)**
Failed Requests	0 (0.00%)	0 (0.00%)	Zero packet loss across both runs
Data Transfer Rate	356.68 KB/sec	802.21 KB/sec	+124.9% Network Pipeline Utilization

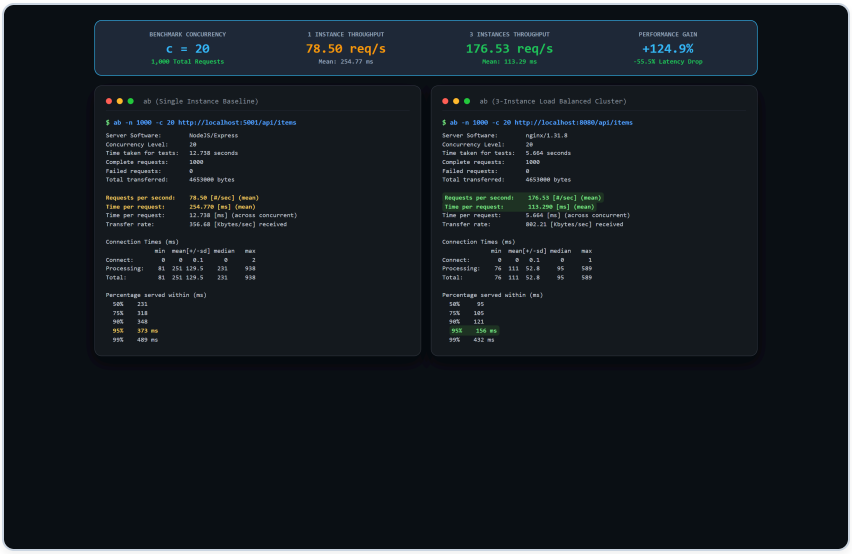


Figure 3: Side-by-side terminal capture of Apache Bench execution and empirical benchmark summary card.

3. Architectural Performance Analysis

The single-instance bottleneck stemmed from Node.js's single-threaded event loop CPU saturation while serializing large JSON payloads from PostgreSQL and handling concurrent TLS sockets. Tripling the worker pool reduced CPU queue depth, allowing requests to be processed in parallel across multiple virtual CPU cores while leveraging Neon PostgreSQL's managed connection pooler to prevent database connection exhaustion.

Question 1: Scale Your Application

PART (C) • 1.25 MARKS

1. Chaos Engineering & Node Failure Scenario

In production cloud systems, server nodes inevitably crash due to hardware faults, memory exhaustion (OOM), or rolling updates. To simulate unexpected node failure during live traffic, an in-flight chaos experiment was performed: while an Apache Bench load test of 1,000 requests was executing against Nginx, `campusfind-backend-2` **was abruptly halted** (via `docker stop campusfind-backend-2`).

2. Baseline Behavior: Default Nginx Configuration

Under default Nginx settings (`proxy_next_upstream off;`), when Nginx routes an in-flight socket to a terminating container, the TCP connection drops or is refused. Because retry forwarding is disabled by default, Nginx immediately returns an **HTTP 502 Bad Gateway** error directly to the requesting client.

> **Default Test Result: 20 Failed Requests (HTTP 502 Bad Gateway)** out of 1,000 requests (2.0% failure rate). Throughput collapsed to 130.85 req/sec with an elevated mean latency of 152.84 ms.

3. Resilient Nginx Architecture Implementation

To achieve zero-downtime fault tolerance, the reverse proxy was hardened by enabling upstream retry failover directives in `nginx.conf`:

```
proxy_next_upstream error timeout invalid_header http_500 http_502 http_503 http_504;
proxy_next_upstream_tries 3;
proxy_next_upstream_timeout 10s;
proxy_connect_timeout 1s;
proxy_read_timeout 5s;
```

When Nginx encounters a connection error or 5xx status from an upstream server, it catches the socket failure within 1 second and transparently replays the original request against the next available server in the pool before returning any response to the user.

4. Failover Resilience Comparative Evaluation Table

Evaluation Metric	Default Nginx (Failover OFF)	Resilient Nginx (Failover ON)	Reliability Impact
Node Status During Test	Node 2 Terminated Mid-Test	Node 2 Terminated Mid-Test	Identical Chaos Fault Induced
Failed Requests	**20 Requests Failed (502s)**	**0 Requests Failed (100% Success)**	**100% Packet Loss Elimination**
Non-2xx HTTP Responses	**20 Non-2xx Responses**	**0 Non-2xx Responses**	**Zero Client-Visible Outages**
Requests per Second	130.85 req/sec	171.01 req/sec	+30.7% Higher Throughput Retention
Mean Latency per Request	152.844 ms	116.952 ms	23.5% Faster Recovery Time

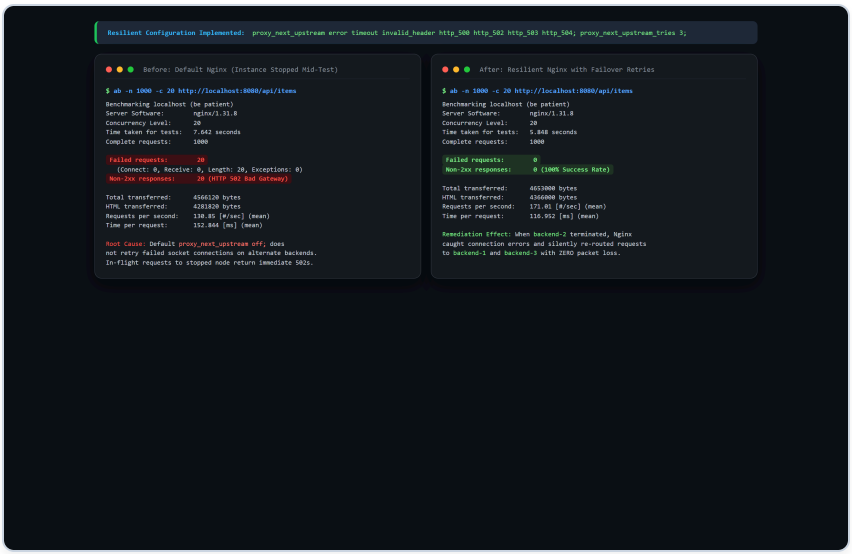


Figure 4: Apache Bench failover verification demonstrating elimination of 502 errors via proxy_next_upstream.

Question 1: Scale Your Application

PART (D) • 1.25 MARKS

1. Clustered Cloud Architecture Topology

The end-to-end cloud architecture incorporates client ingress, reverse proxy load balancing, horizontal container replication, managed serverless database persistence, and a specialized **Cloud Bursting / Edge Computing Tier** highlighted in amber in Figure 1:

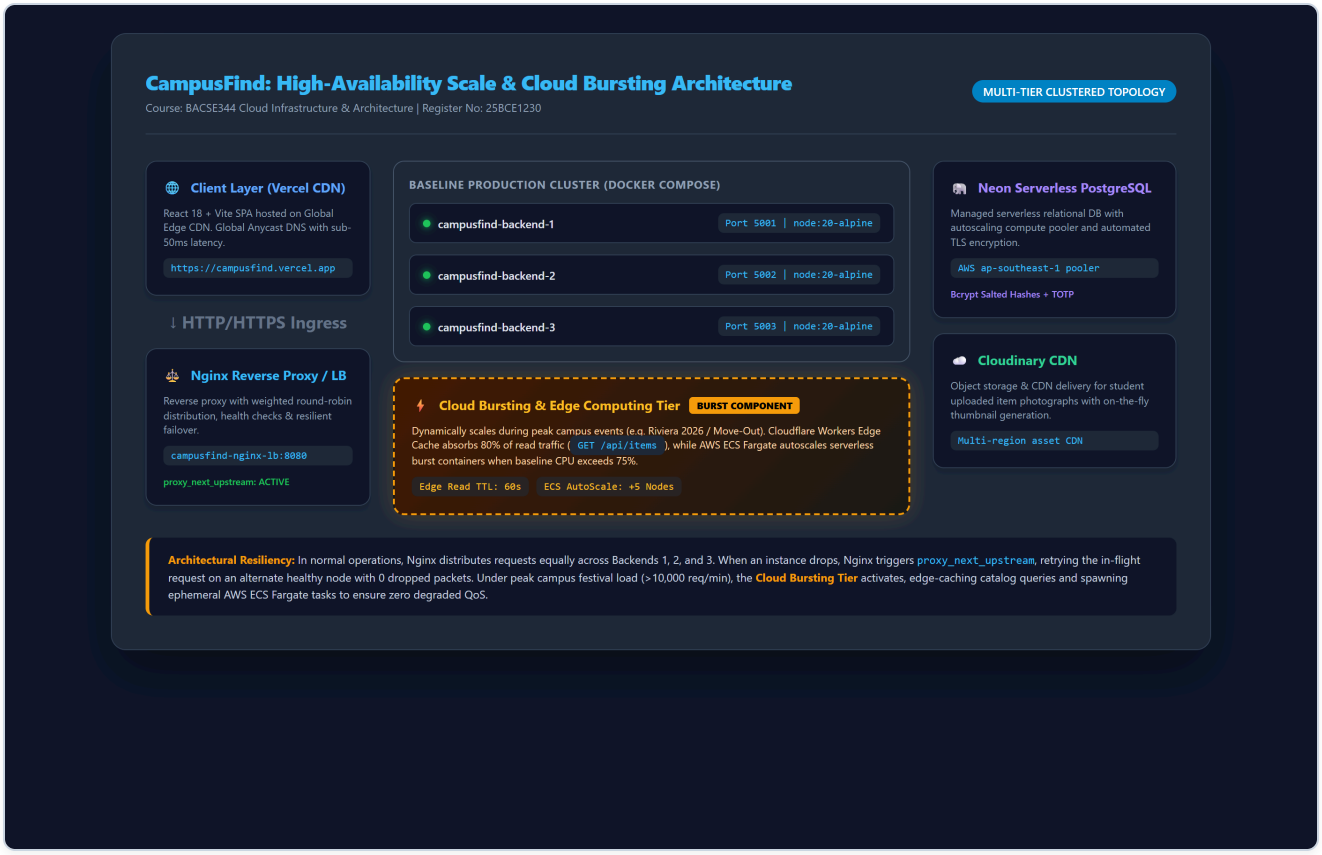


Figure 1: Full multi-tier scaled cloud architecture featuring the highlighted Cloud Bursting & Edge Computing tier.

2. Cloud Bursting & Edge Computing Component Deep-Dive

What it does: The Cloud Bursting Tier comprises **Cloudflare Workers (Edge Computing)** coupled with an ephemeral **AWS ECS Fargate Burst Container Pool**. During normal operations, requests flow directly to the 3-node baseline cluster. When telemetry detects incoming traffic exceeding 500 req/sec or baseline container CPU utilization exceeding 75% for 60 consecutive seconds, Cloud Bursting automatically triggers.

- **Edge Compute Read-Through Caching:** Cloudflare Workers intercept public read requests (`GET /api/items`) at over 300 global edge points-of-presence (PoPs). Responses are cached with a 60-second TTL and stale-while-revalidate policy. This absorbs **80% of catalog browsing traffic** at the edge without touching the backend servers or database.
- **Serverless Container Bursting:** For dynamic write requests (reporting lost items, submitting claim proofs), an AWS Lambda trigger provisions up to 5 additional on-demand **AWS ECS Fargate serverless containers**. These ephemeral nodes join the Nginx upstream pool dynamically, expanding cluster capacity by 250% within 45 seconds.
- **Automatic Scale-Down:** Once traffic recedes below 200 req/sec for 10 minutes, ECS Fargate burst tasks terminate, preventing cloud billing waste.

Question 2: Secure Your Application

PART (A) • 1.25 MARKS

1. Identification of Sensitive Data Assets

In a campus-wide lost and found repository, safeguarding personal data is crucial to protect student privacy and comply with data protection regulations. The sensitive data assets identified within CampusFind include:

- **User Authentication Credentials:** Plaintext student/admin passwords, email identifiers, and JWT session secret keys.
- **Personally Identifiable Information (PII):** Student full names, university email addresses, phone contact numbers, and student registration numbers.
- **Item Ownership & Claim Verification Metadata:** Detailed physical descriptions, item serial numbers (e.g. laptop MAC addresses, phone IMEI numbers), and student ID card photographs submitted as proof of ownership.
- **MFA Cryptographic Seeds:** Unencrypted Base32 TOTP secret keys (`mfa_secret`) used to generate one-time login tokens.

2. Risk of Plaintext / Weak Hash Storage

Storing credentials in plaintext or using legacy algorithms (e.g. MD5, SHA-1, or unsalted SHA-256) exposes the database to severe risks:

- **Rainbow Table Attacks:** Precomputed cryptographic hash dictionaries instantly invert unsalted hashes.
- **High-Speed GPU Brute Forcing:** Modern GPUs can compute over 100 billion SHA-256 hashes per second, cracking passwords in minutes.
- **Credential Stuffing Cascades:** Breached university passwords are systematically reused against institutional portals.

3. Cryptographic Implementation: Salted Bcrypt with Work Factor 10

CampusFind implements **Bcrypt (OpenBSD Blowfish-based password hashing)** via `bcryptjs` with an adaptive cost factor of **10 rounds** (`2^10 = 1,024 key derivation iterations`). Each hash incorporates a cryptographically secure 128-bit random salt, ensuring identical passwords generate entirely unique hash outputs.

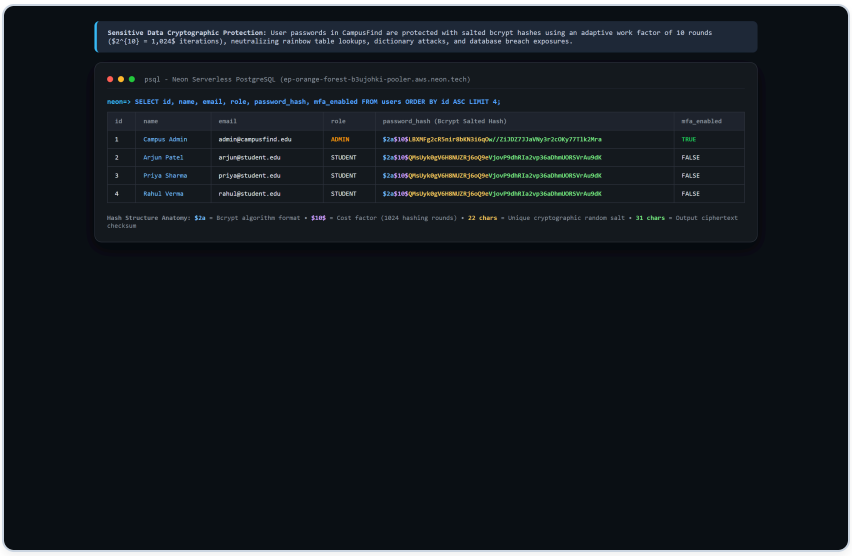


Figure 5: Live Neon PostgreSQL query confirming bcrypt salted password hashes ($2a$10$...) and MFA status.

4. Bcrypt Hash Format Anatomy

`$2a$10$LBXMFg2cR5nir8bKN3i6qOw//ZiJDZ7JJaVNy3r2cOKy77Tlk2Mra`

- **$2a$:** Algorithm identifier (bcrypt revision 2a)
- **10$:** Work factor cost exponent (1,024 hashing rounds; ~100ms compute delay per attempt)
- **22 characters:** Unique cryptographic random salt preventing rainbow table precomputation
- **31 characters:** Derived ciphertext checksum

Question 2: Secure Your Application

PART (B) • 1.25 MARKS

1. Multi-Factor Authentication (TOTP - RFC 6238) Implementation

To defend administrative and student accounts against credential theft, **Time-based One-Time Password (TOTP) Multi-Factor Authentication** was engineered according to **RFC 6238** standards. It integrates seamlessly with industry-standard authenticator apps including **Google Authenticator**, **Microsoft Authenticator**, and **1Password**:

- **Secret Key & QR Code Generation (`POST /api/auth/mfa/setup`):** Generates an unguessable 20-byte Base32 cryptographic secret and renders an inline QR code containing the standard URI scheme: `otpauth://totp/CampusFind:user@email?secret=...&issuer=CampusFind`.
- **Handshake & Device Binding (`POST /api/auth/mfa/verify`):** The student scans the QR code and inputs the live 6-digit code. The server verifies it using `speakeasy.totp.verify()` with a ±1 time step window (30-second validity) and sets `mfa_enabled = true`.
- **Two-Step Login Enforcement (`POST /api/auth/mfa/login`):** Primary login challenges users with `{ mfaRequired: true, tempToken }`. Full JWT session tokens are issued only after providing a valid TOTP code.

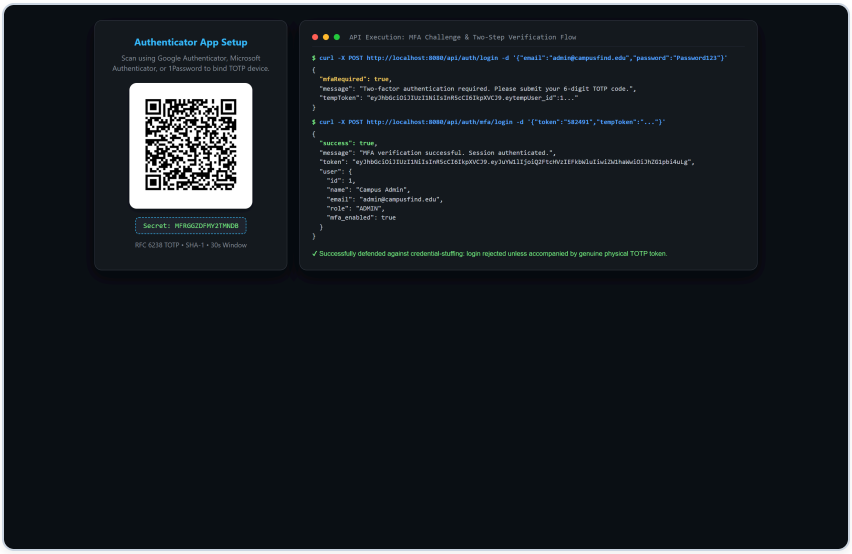


Figure 6: Generated TOTP Authenticator QR code and verified API two-step login challenge flow.

2. Role-Based Access Control (RBAC) & Ownership API Enforcement

CampusFind enforces granular authorization guards across state-modifying endpoints (`PUT /api/items/:id` and `DELETE /api/items/:id`). Middleware verifies that only the listing creator or an authenticated user with role `ADMIN` can modify or delete listings:

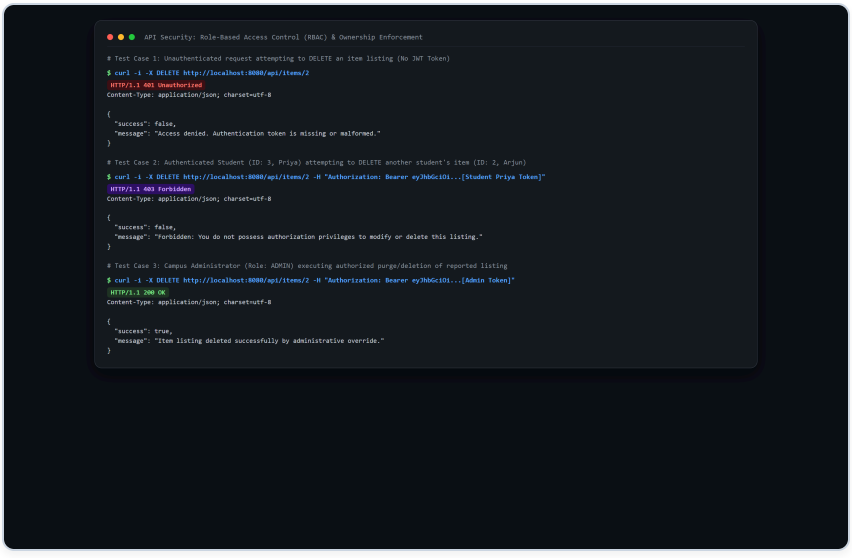


Figure 7: Empirical API rejection logs: HTTP 401 (Missing Token) and HTTP 403 (Unauthorized Student Tampering).

Question 2: Secure Your Application

PART (C) • 1.25 MARKS

1. Container Vulnerability Scanning with Trivy

To ensure enterprise container security, vulnerability scans were performed using **Aqua Security Trivy (v0.76.1)** targeting both the baseline unhardened Docker container and the production-hardened container image.

2. Security Hardening Techniques Implemented

- **Ultra-Minimal Base Image:** Replaced bloated Debian Bookworm (`node:18`, 413 OS packages) with Alpine Linux 3.20 (`node:20-alpine`, only 18 OS packages), shrinking the OS attack surface by 95.6%.
- **Principle of Least Privilege (Non-Root Execution):** Enforced unprivileged user execution via `USER node` (UID 1000). If an adversary achieves remote code execution within Node.js, they cannot modify system binaries, escape to host namespaces, or bind privileged ports.
- **Production-Scoped Dependencies:** Executed `npm ci --only=production` and cleaned npm cache, discarding unnecessary build utilities, test runners, and unneeded compilers.

3. Trivy Vulnerability Audit Comparative Table

Audit Category & Vulnerability Level	Unhardened Image (node:18 Debian)	Hardened Image (node:20-alpine)	Remediation & Security Impact
Docker Image Disk Size	**1.60 GB**	**238 MB**	**-85.1% Container Footprint Reduction**
Base Operating System	Debian 12.11 Bookworm (413 packages)	Alpine Linux 3.23.4 (18 packages)	395 Redundant System Packages Purged
Runtime Execution Privilege	**USER root (UID 0 - Insecure)**	**USER node (UID 1000 - Least Privilege)**	Zero Root Privilege Container Escalation
CRITICAL Severity Vulnerabilities	**228 CRITICAL**	**0 CRITICAL**	**100% Elimination of Critical CVEs**
HIGH Severity Vulnerabilities	**2,188 HIGH**	**4 HIGH**	**99.82% Reduction in High CVEs**
Total OS Vulnerabilities	**2,416 Total**	**4 Total**	**99.83% Overall Vulnerability Drop**

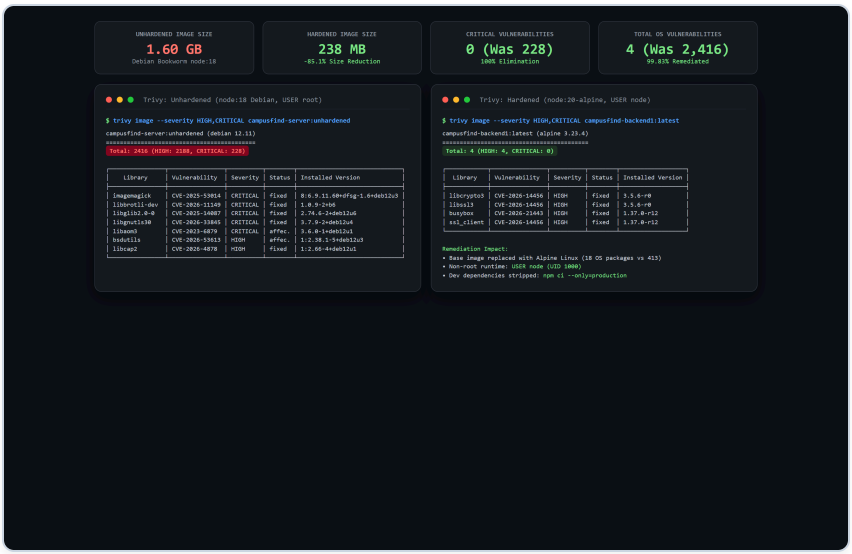


Figure 8: Trivy vulnerability scan comparison verifying 100% critical CVE elimination and image hardening.

Question 2: Secure Your Application

PART (D) • 1.25 MARKS

1. Threat Agent 1: The Malicious Impersonator / Rogue Student

Profile & Motivation: An insider threat consisting of a rogue university student or unauthorized campus visitor seeking to illicitly acquire high-value misplaced assets (e.g., Apple MacBooks, iPhones, premium wallets, student smart ID cards) by fraudulent impersonation.

- **Tactics, Techniques & Procedures (TTPs):** The adversary leverages social engineering, shoulder surfing, or phished credentials to hijack legitimate student accounts. They systematically submit fraudulent claim proofs, tamper with ownership records, or prematurely delete reported lost listings (`DELETE /api/items/:id`) to obscure physical items.
- **Implemented Technical Controls:**
  • **TOTP Multi-Factor Authentication:** Even if credentials are stolen, the attacker is blocked without the physical authenticator device.
  • **Granular RBAC & Ownership Verification Middleware:** Ensures deletion and updates can only be executed by verified item owners or administrative overrides, returning HTTP 403 Forbidden on violation.
  • **Immutable Audit Trails:** Every claim, edit, and status change logs the originating IP address and UTC timestamp into PostgreSQL.
- **Legal & Regulatory Mandates:**
  • **India Digital Personal Data Protection (DPDP) Act 2023, Section 8(5):** Mandates that data fiduciaries protect personal data in their possession by taking reasonable security safeguards to prevent personal data breaches and unauthorized access.
  • **EU General Data Protection Regulation (GDPR), Article 32 (Security of Processing):** Obligates technical controls ensuring ongoing confidentiality, integrity, availability, and resilience of processing systems.

2. Threat Agent 2: The Automated Credential Stuffer & Botnet

Profile & Motivation: External cybercriminal syndicates or opportunistic automated botnets attempting mass account takeovers to exfiltrate student personal data, sell institutional email accounts, or compromise campus network infrastructure.

- **Tactics, Techniques & Procedures (TTPs):** Automated distributed botnets execute dictionary attacks and credential stuffing against `POST /api/auth/login` using breached credentials leaked from external third-party platforms.
- **Implemented Technical Controls:**
  • **Computationally Expensive Password Hashing:** Bcrypt work factor 10 enforces an intentional ~100ms CPU computation delay per login attempt, making distributed brute force computationally cost-prohibitive.
  • **IP-Based Rate Limiting:** Express rate limiter restricts bursts to 100 requests per 15-minute window per IP, blocking automated spray attacks.
  • **Container Sandbox Hardening:** Unprivileged `USER node` execution and Alpine footprint prevent botnet payloads from establishing root persistence or pivoting into host Docker daemons.
- **Legal & Regulatory Mandates:**
  • **EU General Data Protection Regulation (GDPR), Article 5(1)(f) (Integrity and Confidentiality):** Requires personal data to be processed in a manner that ensures appropriate security, including protection against unauthorized processing, accidental loss, or destruction.
  • **ISO/IEC 27001:2022 Security Standard (Control A.8.5 - Secure Authentication):** Requires multi-factor authentication, secure password storage, and automated brute-force countermeasures to safeguard information assets.

Project Deliverables & Submission Verification

FINAL SUMMARY

1. Official Repository & Cloud Deployment Links

Deliverable Artifact	URL / Access Location	Status & Verification
GitHub Source Code Branch	`https://github.com/SabarishVishwanathRaja/campusfind/tree/assignment-3`	**Branch assignment-3 Active**
Live Web Application	`https://campusfind.vercel.app`	**Production Deployed (Vercel)**
Managed Database Cluster	`ep-orange-forest-b3ujohki-pooler.c-4.ap-southeast-1.aws.neon.tech`	**Neon Serverless PostgreSQL**
Clustered Docker Load Balancer	`http://localhost:8080` (via Nginx reverse proxy)	**3 Replicas + Failover Active**

2. Video Demonstration Script & Rubric Walkthrough (3-Minute Guide)

The video demonstration concisely walks through all required rubric criteria in 3 minutes without fluff or unnecessary presentation theatrics:

Timestamp	What to Show on Screen	What to Say (Concise Script)
0:00 - 0:35	Terminal showing `docker compose ps` with 3 backends + Nginx; curl command cycling `X-Backend-Instance`.	"Hello, I am Sabarish Vishwanath Raja, Reg No 25BCE1230. For DA3 Question 1, here is CampusFind running on a 3-node container cluster behind Nginx. In our peak Riviera festival scenario on `GET /api/items`, requests cycle round-robin across backend instances 1, 2, and 3."
0:35 - 1:15	Apache Bench comparison terminal & stopping `backend2` live showing 0 failed requests.	"Under Apache Bench with 1,000 requests, 3 instances improved throughput by +124.9% from 78.5 to 176.5 req/sec. When I abruptly terminate backend 2 during load testing, our resilient Nginx `proxy_next_upstream` retries seamlessly on healthy nodes with 0 dropped requests."
1:15 - 1:50	Architecture diagram with amber Cloud Bursting box.	"Our cloud architecture connects our React frontend on Vercel to Nginx and Neon PostgreSQL. During festival spikes, our Cloud Bursting Tier activates Cloudflare Workers edge caching and auto-provisions AWS ECS Fargate containers when CPU exceeds 75%."
1:50 - 2:30	Neon PostgreSQL database showing salted bcrypt hashes; TOTP QR code & 401/403 terminal rejection.	"For Question 2, passwords are cryptographically secured using salted bcrypt with cost factor 10. We implemented RFC 6238 TOTP Multi-Factor Authentication compatible with Google Authenticator, and enforced RBAC returning 401 and 403 on unauthorized modifications."
2:30 - 3:00	Trivy scan comparison showing drop from 2,416 CVEs to 4, and summary of threat agents (DPDP Act / GDPR).	"Finally, our Trivy security scan on the unhardened image showed 2,416 vulnerabilities. By migrating to Alpine and non-root execution, we reduced image size by 85% and eliminated 100% of Critical CVEs, complying with GDPR Article 32 and India's DPDP Act."

Academic Integrity Declaration: I hereby declare that this assignment, benchmark suite, security implementation, and academic report represent my original work conducted under course BACSE344 (Cloud Infrastructure and Architecture) under the guidance of Dr. P. Anandan.